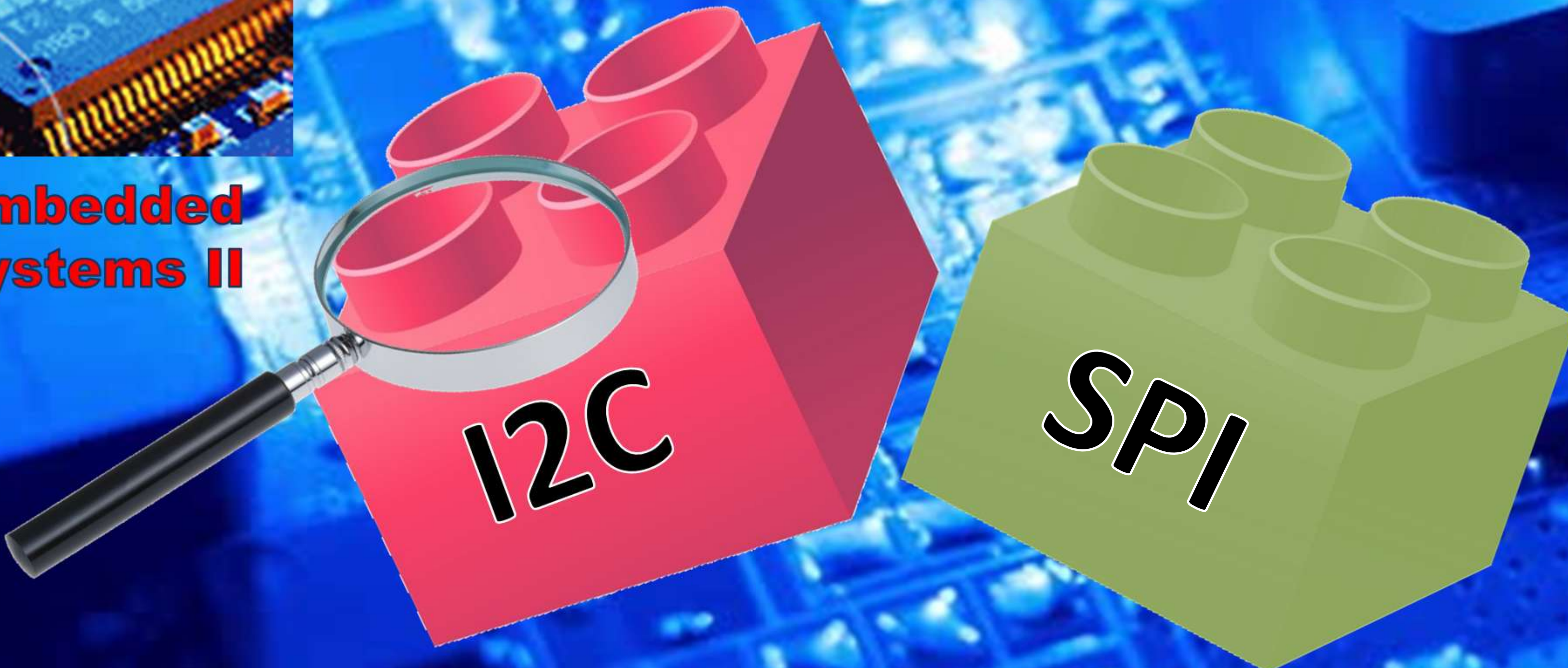
EEE3096S

Embedded Systems II



Embedded Communications 2/3

L13

Embedded Systems II

Lecturer Best :)

Electrical Engineering
University of Cape Town

Outline of Lecture

- Getting into the SPI and I2C serial protocols

Plan for 03 Sept:

- Training for I2C

Acknowledgement: these slides adapted from ESII 2021 slides by J. Wyngaard.

ES Communications Training Plan

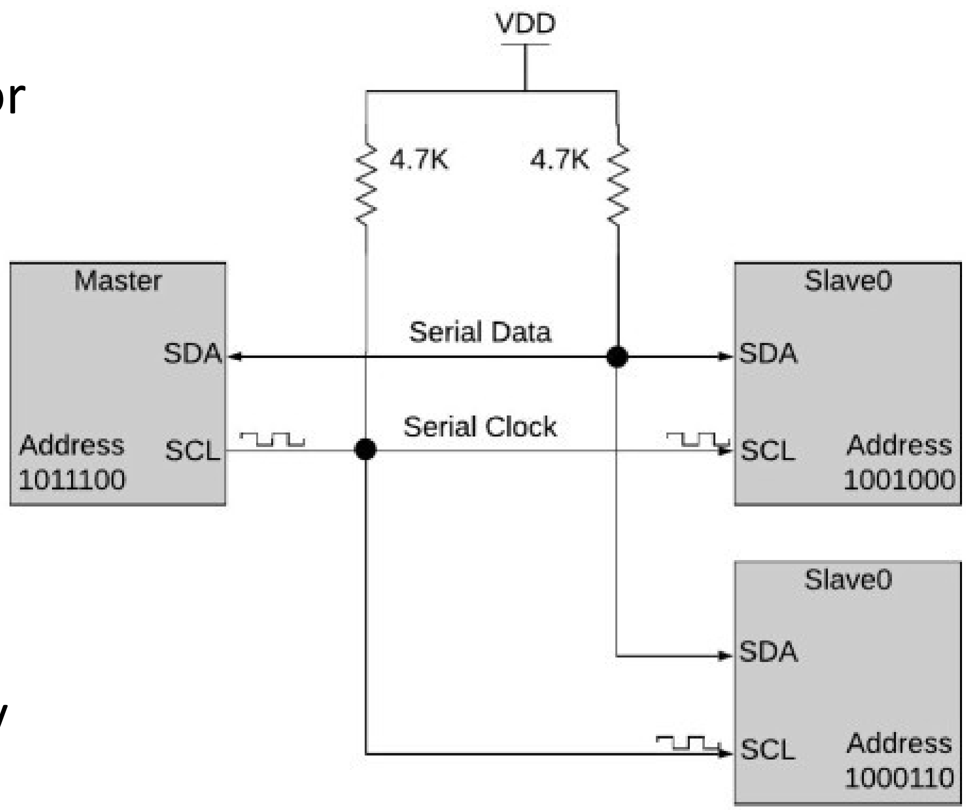
- Comms 1:
  - Intro: Serial vs Parallal, Synch/Asynch
- Comms 2:
  - I2C & SPI
  - PWMs & Peripherals
- Comms 3:
  - Timers
  - RS232 & RS485
  - Briefly, some other interface protocols
  - {Brief Introduction to Wireless Communications – if time permits}

The I2C Protocol

I2C Protocol Overview

I2C = Inter-Integrated Circuit, aka “eye squared” (I2C)

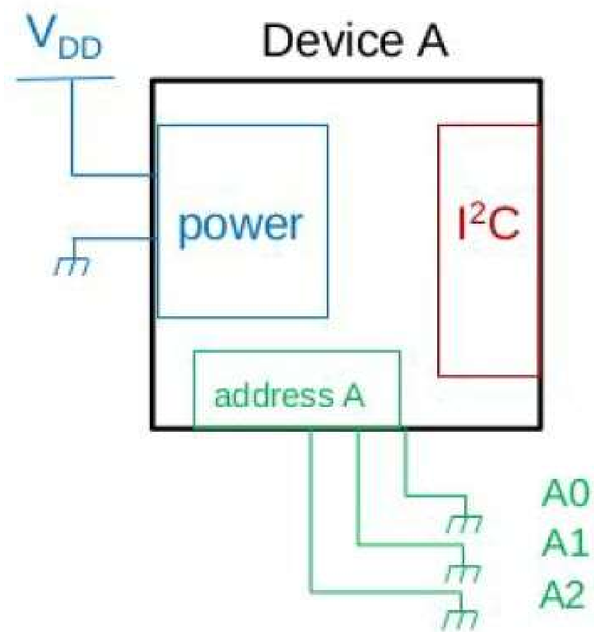
- A communication protocol Invented by Philips Semiconductor in 1989
- **Half duplex** (both ends can send but not simultaneously)
- **Serial** (bits sent sequentially one at a time)
- **Synchronous** (uses shared clock)
- **Simple**, robust, low cost
- Easy to use when you need communication between a chain of devices and a microprocessor / microcontroller



- 2 wires:
  - **SDA:** Serial Data Line
  - **SCL:** Serial Clock Line
  - **SDA:** state changes when SCL is low

I2C Protocol Overview

- Supports multiple master and multiple slaves
- Widely used for short distance on PCB / nearby off-board communications
  (Eg: RTC, Temp sensors, ADC, EEPROM...)
- Board layout is simple
- Many libraries in many languages are readily available
- Verification of every byte is built in:
  - An acknowledge (ACK) signal is built into the protocol after every data word
- Speeds:
  - Standard mode: 100 kbit/s
  - Fast mode: 400 kbit/s
  - High speed: 3.4 Mbit/s



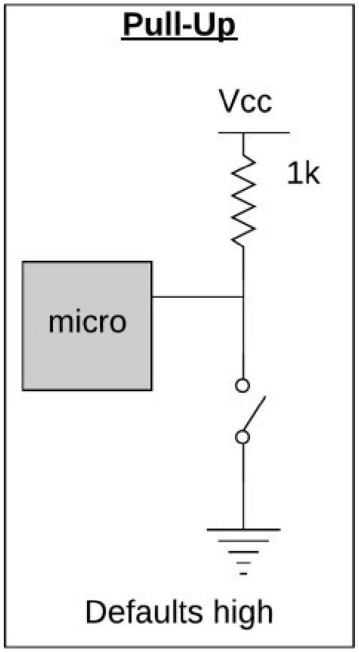
- Often some bits of the address will be set in hardware while others can be defined by the PCB designer.

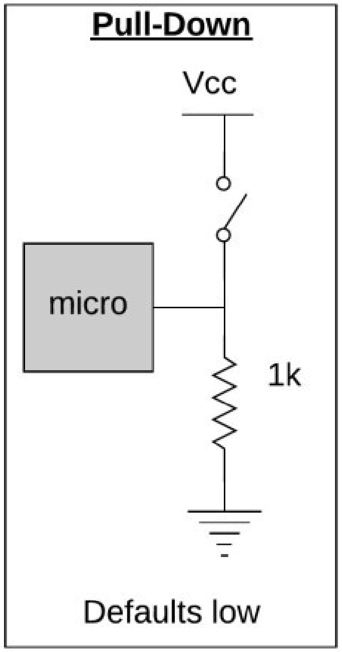
e.g.: A2, A1, A0 are held to ground in the PCB = '000'
A6, A5, A4, A3 are set in the IC device

Recap: Integrated Circuit Input/Outputs

Digital logic can be High / Low / Floating (undefined)

- Pull up/down resistors remove the chance of floating/undefined values by creating a default so the state of a pin is always known
- Some ICs include these pull up/down resistors internally (check the datasheet)
- Resistor value depends on pin impedance and leakage current

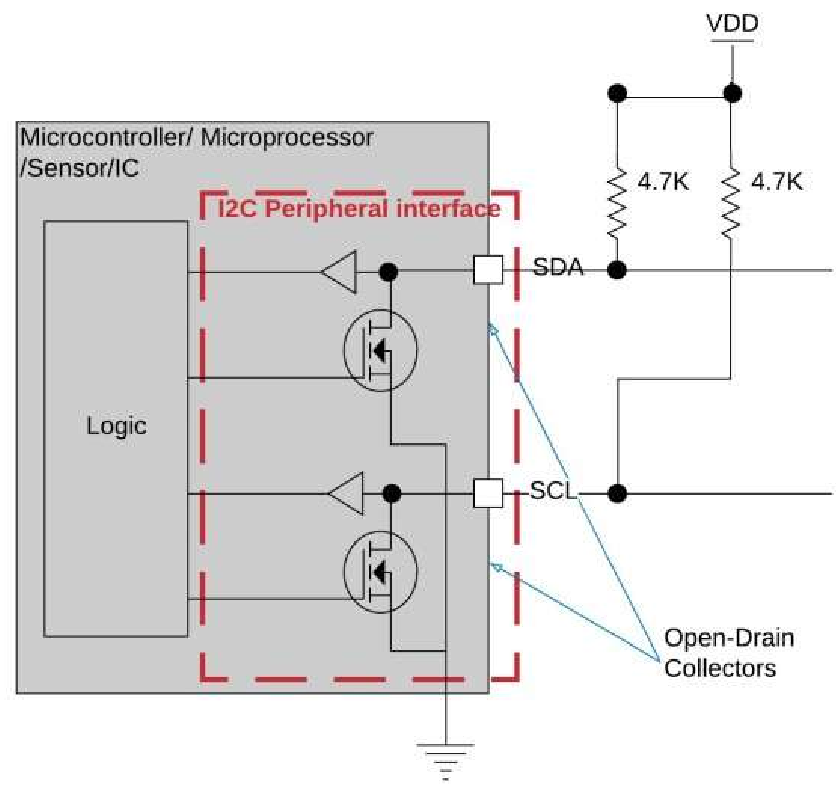




I2C Hardware

I2C interface pins will have an open drain collector on both the clock and data pins.

- A 4.7k is a standard recommendation for pull-up resistors to use.
- Bus signals always default to logic high and so must be actively pulled low for operation
  - If a devices dies the lines will return high and be available for other devices to continue functioning
  - If needed a slave can hold the clk low to reduce frequency
  - Different VCCs on the same lines can exist (provided other devices can tolerate higher V)

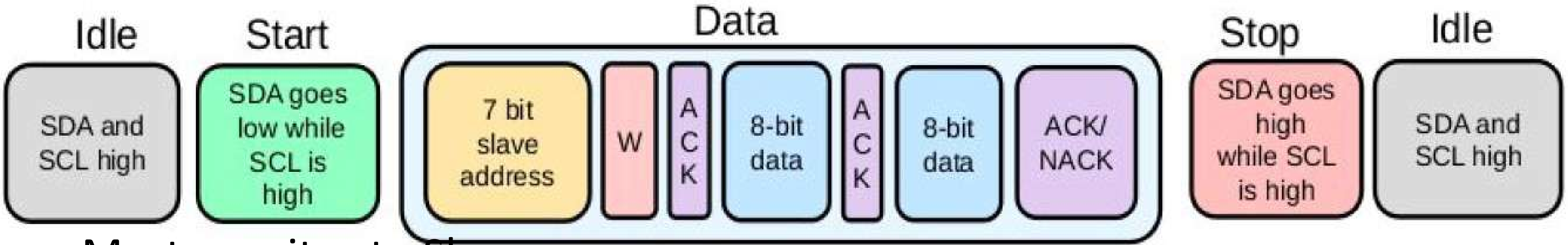


I2C Protocol

Sequence of actions performed when communicating via I2C:

1. **Idle state:** SDA & SCL Signals HIGH *//Default state for SCL and SDA is HIGH*
2. **Start command:** SDA transitions from HIGH to LOW while SCL is HIGH. *//Pulling low is an active action*
3. **Stop command:** SDA transitions from LOW to HIGH while SCL is high *//Release control of line*
4. **Read:** Transmitter leaves SDA to stay HIGH (logic 1) after an address
5. **Write:** Transmitter pulls SDA LOW (logic 0) after an address
6. **Acknowledge (ACK):** Receiver actively pulls SDA LOW
7. **Not Acknowledge (NACK):** Receiver doesn't pull SDA LOW, defaults to HIGH

I2C Protocol: Master writes to Slave



- Master writes to Slave:

1. Both lines idle (HIGH)
2. Master transmits Start: SDA pulled LOW while SCL is high by master
3. Master transmits 7-bit Slave address to indicated which slave should listen to the data arriving
4. Master transmits Write bit: SDA pulled LOW for 1 bit by master or a read bit SDA pulled LOW for 1 bit
5. Receiving Slave (who was addressed) transmits ACK (acknowledge) bit: SDA is actively pulled LOW for 1 bit by the addressed slave, all other slaves leave line high. If address was invalid line is left high by all = NACK (not acknowledge)
6. Master transmits 8-bits of data, this is usually the address
7. Receiving Slave transmits ACK: SDA pulled LOW to acknowledge transmission
8. Master transmits next 8-bits of data
9. Master transmits Stop command to indicate end of communication with that slave (lets SDA stay high while SCL HIGH)

I2C Protocol: Master writes to Slave

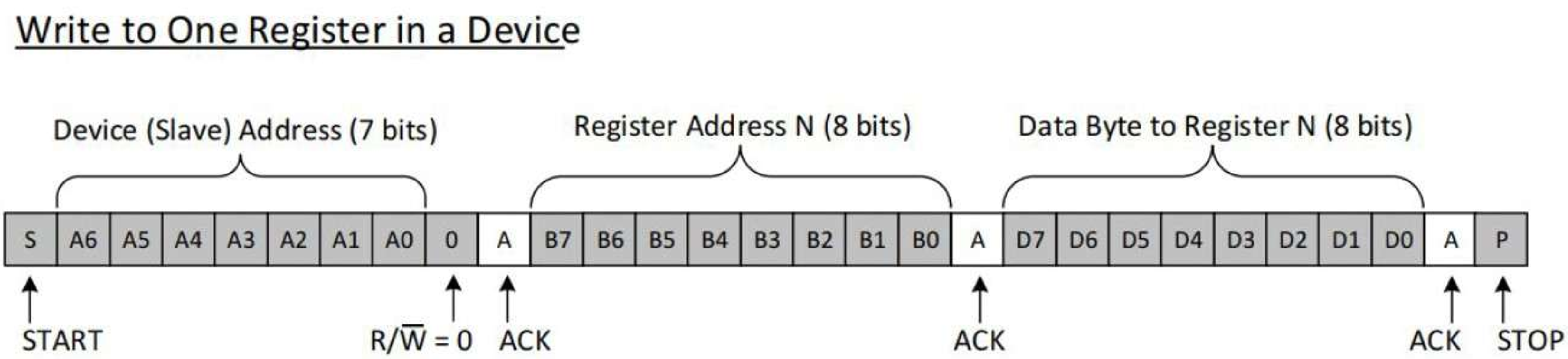


Figure 8. Example I²C Write to Slave Device's Register

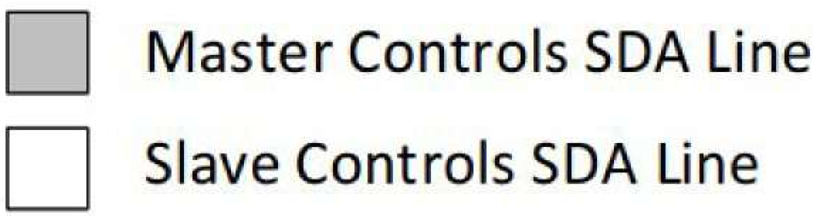
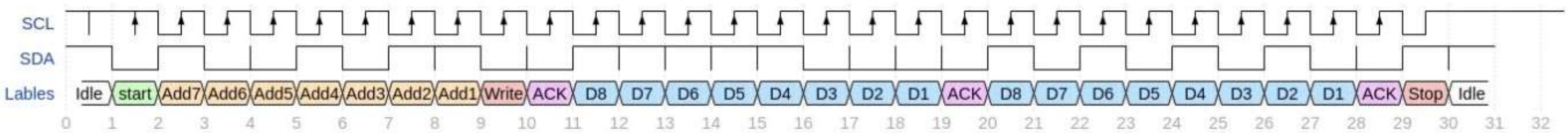


Illustration showing sequence of bits sent on the I2C SDA line, which bits are driven by the master (gray block) and which are driven by slave (white blocks).

I2C Protocol: Master writes to Slave

Illustration of SCL (clock) and SDA (data) lines
SCL positive edge indicates SDA is ready to be read



I2C Arbitration and clock stretching

Arbitration =
With multiple masters on a bus, each monitors for idle (both lines high) and only uses the bus when it is free.
If two masters attempt to use the bus simultaneously, the first to pull SDA low while trying to transmit a '0' will 'win' and the other will back off.

Clock stretching =
Any device can hold SCL low, if a device wishes to slow the clock it can do so by simply holding SCL low longer than is otherwise anticipated.

I2C in context

Key Disadvantage:

- Frame overhead (address and ACK bits) costs time
- Hardware complexity increases as master/slave devices are added

Key Advantages:

- Only 2 wires
- Ack facilitates reliable operation
- Easy support for multiple Vdd levels (assuming all ICs can tolerate highest level)